

Übung Cybersicherheit

Übungsblatt 11: Einführung in Software-Exploits

Christian Niesler
Tristan Löding

Fakultät für Informatik
Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>
Universität Duisburg-Essen

Aufgabe 1: Hello World (x86-32)

GDB Output

```
1 mov    eax,0x4
2 mov    ebx,0x1
3 mov    ecx,0x804901f
4 mov    edx,0xd
5 int     0x80
6 mov    eax,0x1
7 xor    ebx,ebx
8 int     0x80
9 dec    eax
10 gs ins BYTE PTR es:[edi],dx
11 ins    BYTE PTR es:[edi],dx
12 outs   dx,DWORD PTR ds:[esi]
13 and    BYTE PTR [edi+0x6f],dl
14 jb     8049095 <_start+0x95>
15 and    DWORD PTR fs:[edx],ecx
```

System Calls (32-bit)

Interface zu **Kernel Funktionen** für Benutzerprozesse

Zugriff auf Hardware

Prozessmanagement, Speichermanagement

Assembly Instruktion: *int 0x80* (32-bit), *syscall* (64-bit)

Parameter in Registern, **Rückgabewert in Register *eax***

Number	Name	eax	ebx	ecx	edx	esi	edi
1	sys_exit	0x01	int error_code	-	-	-	-
4	sys_write	0x04	unsigned int fd	char *buf	size_t count	-	-
			...				

```
1 mov  eax,0x4
2 mov  ebx,0x1
3 mov  ecx,0x804901f
4 mov  edx,0xd
```

```
5 int  0x80
```

```
6 mov  eax,0x1
7 xor  ebx,ebx
```

```
8 int  0x80
```

```
9 dec  eax
```

```
10 gs ins BYTE PTR [ecx],eax
```

```
11 ins  BYTE PTR [es:edi],eax
```

```
12 outs DWORD PTR [ds:ecx],eax
```

```
13 and  BYTE PTR [ecx,0x6f],0
```

```
14 jb  8049095 < _start
```

```
15 and  DWORD PTR [ecx],ecx
```

SYS_WRITE

SYS_EXIT

Das Programm
terminiert an dieser
Stelle

Rückgabewert in
eax

TLDR: Aufgabe 1

Number	Name	eax	ebx	ecx	edx	esi	edi
1	sys_exit	0x01	int error_code	-	-	-	-
4	sys_write	0x04	unsigned int fd	char *buf	size_t count	-	-
...							

```
1 mov    eax,0x4                ; SYSCALL WRITE
2 mov    ebx,0x1                ; Schreibe zum Standard Output
3 mov    ecx,0x804901f          ; Adresse zum String "Hello World\n"
4 mov    edx,0xd                ; Anzahl der zu schreibenden Zeichen
5 int     0x80                  ; WRITE(1, "Hello World\n", 12);
6 mov    eax,0x1                ; SYSCALL EXIT
7 xor    ebx,ebx                ; Exit Code 0
8 int     0x80                  ; SYSCALL EXIT(0);
```

Aufgabe 2: Data-Only Attack

```
1 // password kann beliebig groß sein
2 int pw_check(char *password) {
3     int auth = 0;
4     char pw_buffer[16] = {0};
5     strcpy(pw_buffer, password);
6     if (strcmp(pw_buffer, "cyssec") == 0)
7         auth = 1;
8     return auth;
9 }
```



Was ist der Fehler?

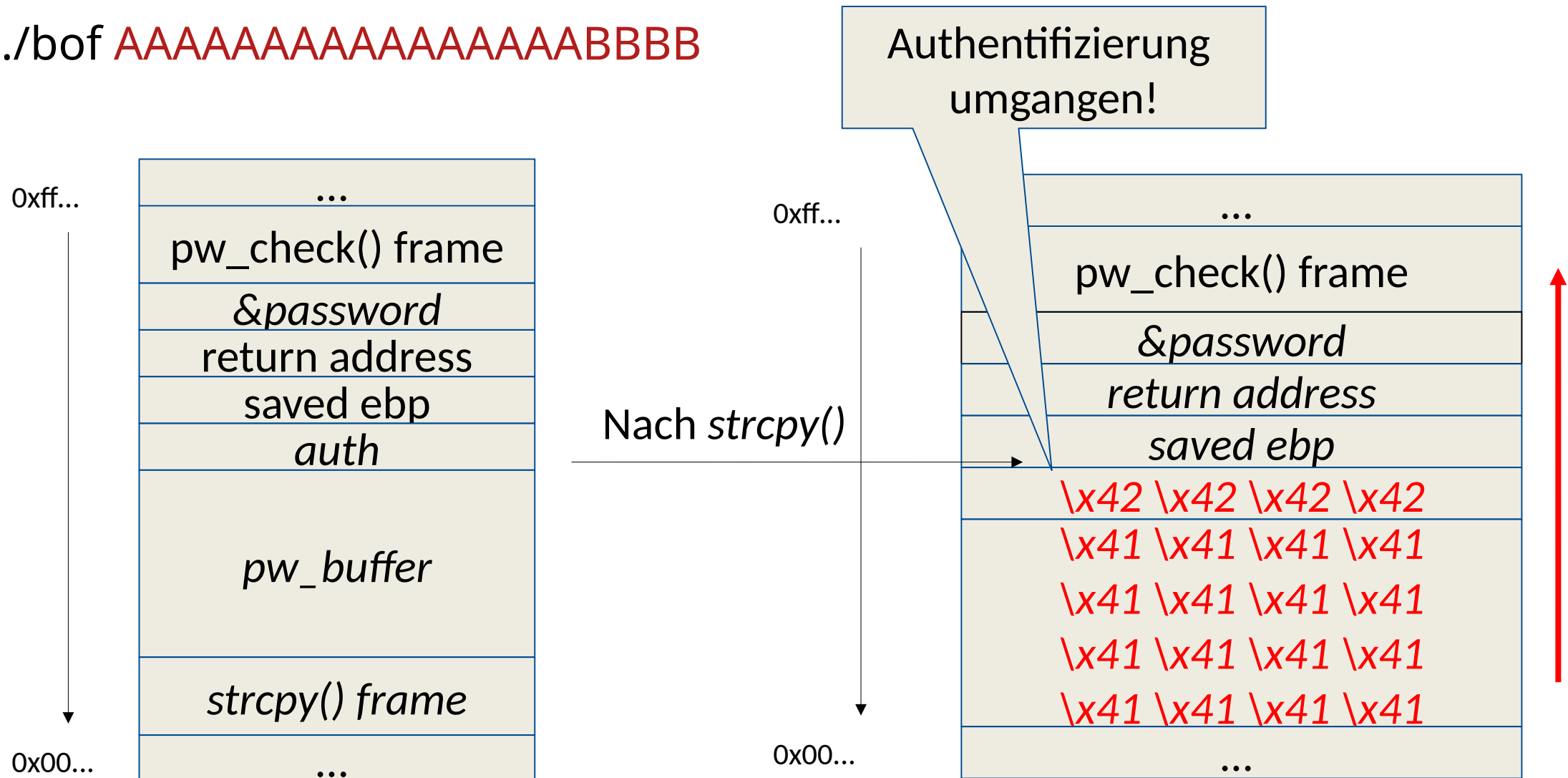
Zeile 5: strcpy(pw_buffer,password)

Kopiert password (inklusive 0-byte) nach
pw_buffer

Was passiert, wenn die Eingabe größer 16 ist?

Stack Frame

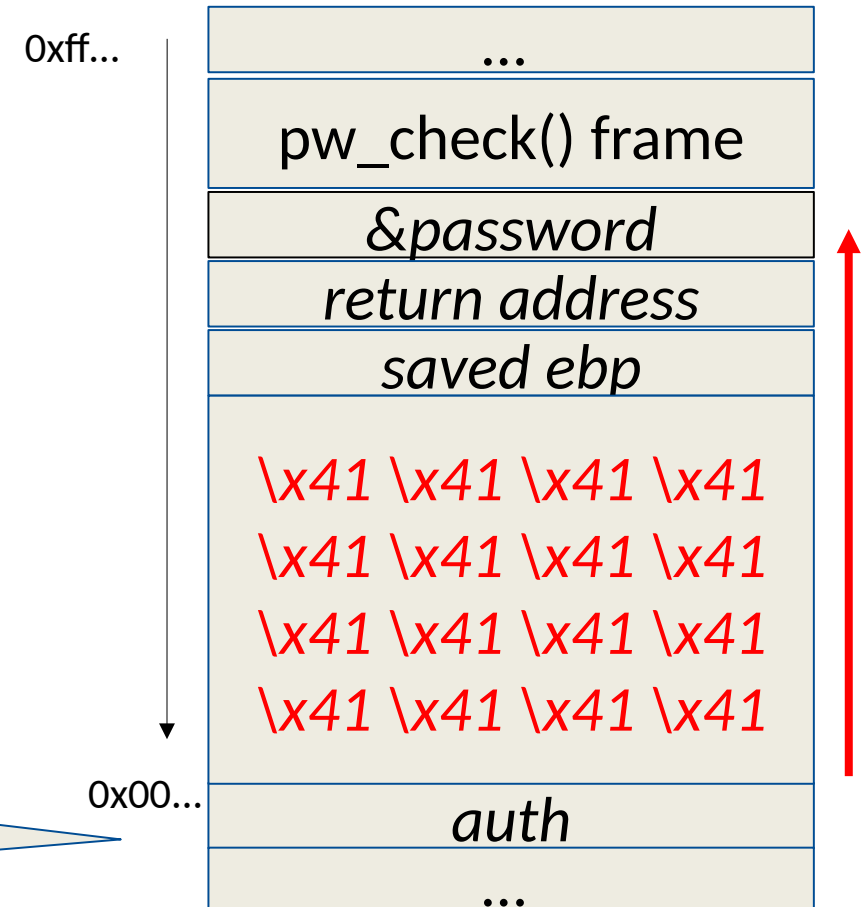
\$./bof AAAAAAAAAAAAAAAAAABBBB



TLDR: Aufgabe 2

```
1 // password kann beliebig groß sein
2 int pw_check(char *password) {
3     int auth = 0;
4     char pw_buffer[16] = {0};
5     + if (strlen(password) > 16) exit(0);
6     strcpy(pw_buffer, password);
7     if (strcmp(pw_buffer, "cyssec") == 0)
8         auth = 1;
9     return auth;
10 }
```

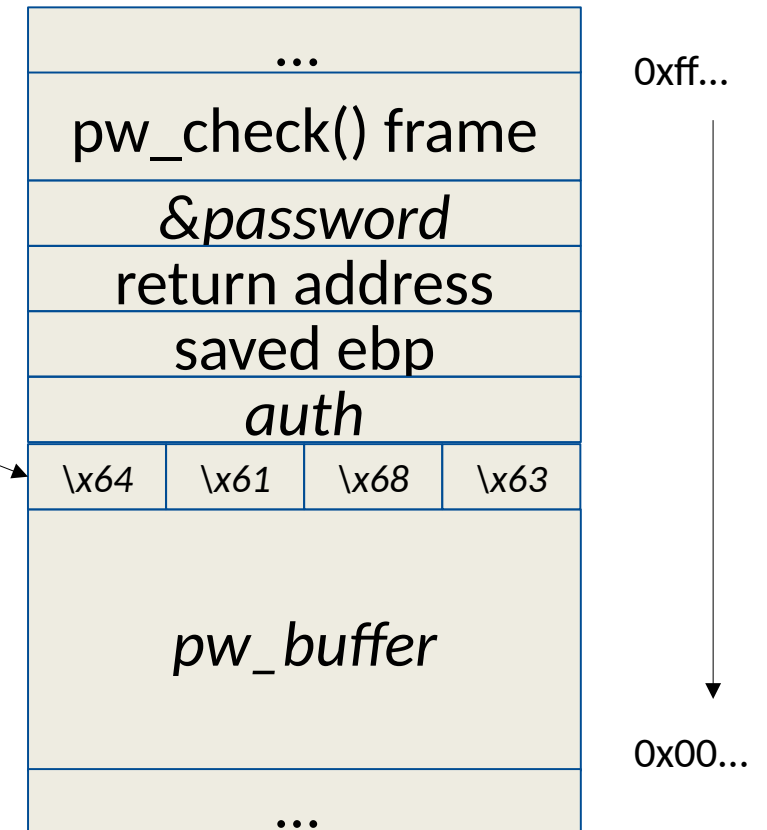
Variable *auth* ist
nicht betroffen



Bonusaufgabe: Canary

Das Programm terminiert bei zu lange Eingaben
Das Programm erkennt Buffer Overflows

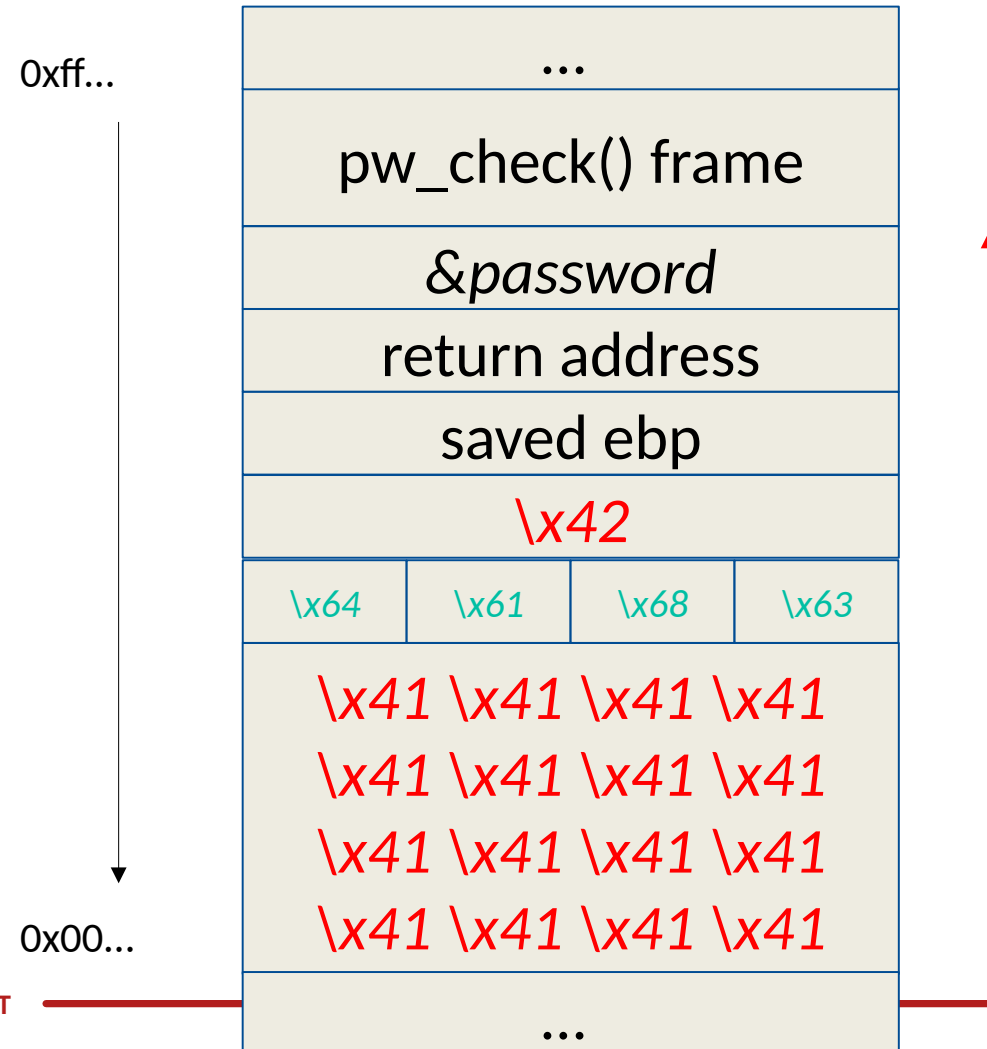
```
mov  BYTE PTR [ebp-0xd],0x64 ; d
mov  BYTE PTR [ebp-0xe],0x61 ; a
mov  BYTE PTR [ebp-0xf],0x68 ; h
mov  BYTE PTR [ebp-0x10],0x63 ; c
```



Am Ende der Funktion wird überprüft, ob
sich die **lokalen Variablen** veränderten!

Lösung

\$./canary AAAAAAAAAAAAAAAAAAChadB



Übung Cybersicherheit SoSe 2025

Vielen Dank für Ihre Aufmerksamkeit

Christian Niesler
Tristan Löding

Fakultät für Informatik
Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>
Universität Duisburg-Essen